AFRILANG Stdlib

std/c/mllinear

Français. Bibliothèque AFRILANG ‘std/c/mllinear‘ (niveau complex, catégorie complex). Elle expose 6 fonction(s) : dot, predict, mse, gradientStep, trainEpochs, sigmoid.
‘import ”std/c/mllinear”‘ · ‘use mllinear‘
- ‘dot(a list number, b list number) → number‘ - ‘predict(weights list number, features list number, bias number) → number‘ - ‘mse(weights list number, X list number, y list number, dims number) → number‘ - ‘gradientStep(weights list number, X list number, y list number, lr number, dims number) → list number‘ - ‘trainEpochs(weights list number, X list number, y list number, lr number, epochs n **English.** AFRILANG library ‘std/c/mllinear‘ (tier complex, category complex). Exports 6 function(s): dot, predict, mse, gradientStep, trainEpochs, sigmoid.
‘import ”std/c/mllinear”‘ · ‘use mllinear‘
- ‘dot(a list number, b list number) → number‘ - ‘predict(weights list number, features list number, bias number) → number‘ - ‘mse(weights list number, X list number, y list number, dims number) → number‘ - ‘gradientStep(weights list number, X list number, y list number, lr number, dims number) → list number‘ - ‘trainEpochs(weights list number, X list number, y list number, lr number, epochs number, dims n

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/mllinear‘ (niveau complex, catégorie complex). Elle expose 6 fonction(s) : dot, predict, mse, gradientStep, trainEpochs, sigmoid.

‘import ”std/c/mllinear”‘ · ‘use mllinear‘

```
- `dot(a list number, b list number) → number`
- `predict(weights list number, features list number, bias number) → number`
- `mse(weights list number, X list number, y list number, dims number) → number`
- `gradientStep(weights list number, X list number, y list number, lr number, dims
    number) → list number`
- `trainEpochs(weights list number, X list number, y list number, lr number, epochs
    number, dims number) → list number`
- `sigmoid(x number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mllinear"
use mllinear
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 dot(a list number, b list number) → number•
 predict(weights list number, features list number, bias number) → number•
 mse(weights list number, X list number, y list number, dims number) → number•
 gradientStep(weights list number, X list number, y list number, lr number, dims
    number) → list•
 trainEpochs(weights list number, X list number, y list number, lr number, epochs
    number, dims number) → list•
 sigmoid(x number) → number
```

4. Exemples d’utilisation

```
import "std/c/mllinear"
use mllinear
```

```
create result = dot(/* args */)
say result
```

```
create result = predict(/* args */)
say result
```

```
create result = mse(/* args */)
say result
```

```
create result = gradientStep(/* args */)
say result
```

```
create result = trainEpochs(/* args */)
say result
```

```
create result = sigmoid(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/mllinear"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module mllinear

```
function dot(a list number, b list number) returns number
end
```

```
function predict(weights list number, features list number, bias number) returns
    number
end
```

```
function mse(weights list number, X list number, y list number, dims number) returns
    number
end
```

```
function gradientStep(weights list number, X list number, y list number, lr number,
    dims number) returns list number
end
```

```
function trainEpochs(weights list number, X list number, y list number, lr number,
    epochs number, dims number) returns list number
end
```

```
function sigmoid(x number) returns number
end
end
```

English

1. Overview

AFRILANG library ‘std/c/mllinear‘ (tier complex, category complex). Exports 6 function(s): dot, predict, mse, gradientStep, trainEpochs, sigmoid.

‘import ”std/c/mllinear”‘ · ‘use mllinear‘

```
- `dot(a list number, b list number) → number`
- `predict(weights list number, features list number, bias number) → number`
- `mse(weights list number, X list number, y list number, dims number) → number`
- `gradientStep(weights list number, X list number, y list number, lr number, dims
    number) → list number`
- `trainEpochs(weights list number, X list number, y list number, lr number, epochs
    number, dims number) → list number`
- `sigmoid(x number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mllinear"
use mllinear
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 dot(a list number, b list number) → number•
 predict(weights list number, features list number, bias number) → number•
 mse(weights list number, X list number, y list number, dims number) → number•
 gradientStep(weights list number, X list number, y list number, lr number, dims
    number) → list•
 trainEpochs(weights list number, X list number, y list number, lr number, epochs
    number, dims number) → list•
 sigmoid(x number) → number
```

4. Usage examples

```
import "std/c/mllinear"
use mllinear
```

```
create result = dot(/* args */)
say result
```

```
create result = predict(/* args */)
say result
```

```
create result = mse(/* args */)
say result
```

```
create result = gradientStep(/* args */)
say result
```

```
create result = trainEpochs(/* args */)
say result
```

```
create result = sigmoid(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/mllinear"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module mllinear

```
function dot(a list number, b list number) returns number
end
```

```
function predict(weights list number, features list number, bias number) returns
    number
end
```

```
function mse(weights list number, X list number, y list number, dims number) returns
    number
end
```

```
function gradientStep(weights list number, X list number, y list number, lr number,
    dims number) returns list number
end
```

```
function trainEpochs(weights list number, X list number, y list number, lr number,
    epochs number, dims number) returns list number
end
```

```
function sigmoid(x number) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mllinear"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/mllinear/

Source (extrait)

```
module mllinear

function dot(a list number, b list number) returns number
end

function predict(weights list number, features list number, bias number) returns
    number
end

function mse(weights list number, X list number, y list number, dims number) returns
    number
end

function gradientStep(weights list number, X list number, y list number, lr number,
    dims number) returns list number
end

function trainEpochs(weights list number, X list number, y list number, lr number,
    epochs number, dims number) returns list number
end

function sigmoid(x number) returns number
end
end
```